Superposition Engine

Connected Information

One bit is the smallest amount of connected information in pairwise exchange interactions[1]:

> "multi-information alone does not tell us how much of the nonindependence among N variables is intrinsic to the full N variables and how much can be explained from pairwise, triple, and higher order interactions. For example, if the $x_i's$ are binary variables or equivalently Ising spins σ_i, and if the full distribution $P(\{\sigma_i\})$ is a conventional Ising model with pairwise exchange interactions, then in an obvious sense there is nothing 'new' to learn by observing triplets of spins that cannot be learned by looking at all the pairs. On the other hand, if σ_3 is formed as the exclusive OR (XOR) of the variables σ_1 and σ_2, then the essential structure of $P(\sigma_1, \sigma_2, \sigma_3)$ is contained in a three-spin interaction; if σ_1 and σ_2 are chosen at random as inputs to the XOR, then all pairwise mutual informations among the σ_i will be zero, although the multi-information will be 1 bit."

	$I(\sigma_1;\sigma_2;\sigma_3)$	$I_C^{(3)}$	$I_C^{(2)}$	$I(\sigma_1;\sigma_2)$	$I(\sigma_1;\sigma_3)$	$I(\sigma_2;\sigma_3)$	R (or I_3)
σ_1, σ_2 — AND → σ_3	0.8113	0	0.8113	0	0.3113	0.3113	-0.1887
σ_1, σ_2 — OR → σ_3	0.8113	0	0.8113	0	0.3113	0.3113	-0.1887
σ_1, σ_2 — XOR → σ_3	1	1	0	0	0	0	-1

Figure 1: Multi (connected) information (adapted from Reference 1)

Interpretation: AND & OR rows show fractions of connected information, indicating unresolved probabilities. The XOR row is different: +1's, 0's and −1's for the connected (mutual) information. Whole units imply the conservation of information. Negative redundancy (−1) in the R column implies the reversal of causality. This is the symmetry we expect from EQ/CQ (Equal Quantities/Conserved Quantities). Columns 3 and 4 show connected information (across triples). Columns 5, 6, 7 are pairwise mutual information – zero because all the pairs do not 'own' the connected information, only the triples (and larger) do, which is indicated in the 2nd & 3rd column.

MORTs (Morphism pORTS) – Open Petri Net Model

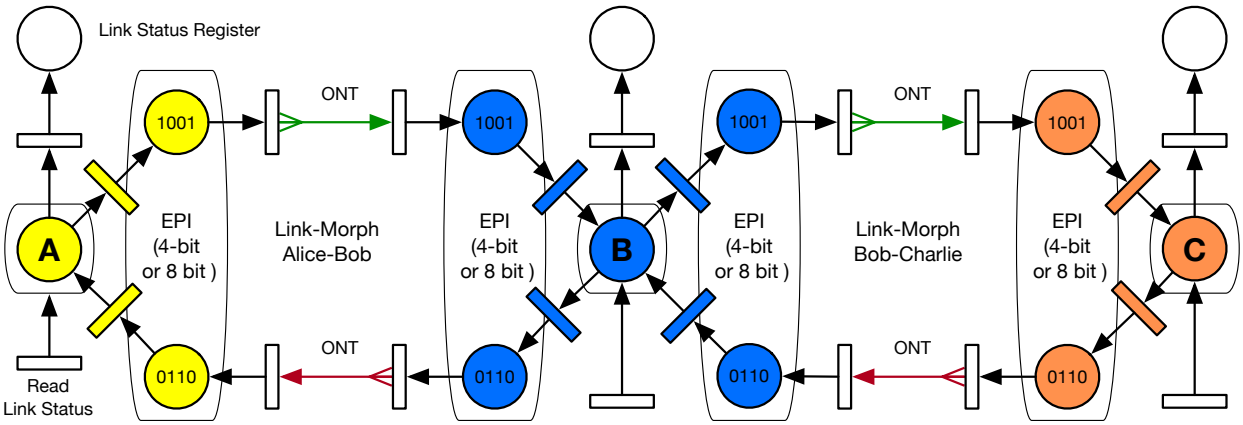

Figure 2: 3 cells, 2 Links (MORTS) between Alice, Bob and Charlie

Figure 2 Shows the basic Petri Net model for three cells (Alice, Bob, and Charlie). The EPI information is represented by the last place before the packet goes out on the wire, and the first place where a packet comes in on the wire. The link status is driven by the XOR function, further limiting its epistemic knowledge to simply whether the link is down, or is up and live. The white boxes and places below and above the link status forms a destructive read.

Rather than choose σ_1, σ_2 from some artificial distribution as inputs to the XOR to get σ_3, we note there is a race condition between A, B, and C, and instead of unknowable probabilities (σ_i), we can instead 'capture' one of a set of compatible orderings specified by an allowed multiway schedule. When one of them is observed locally, the Substructure Superposition Engine (SSE) can be programmed to trigger the completion of the measurement, resolving the bit-commitment to a zero or one.

[1] Schneidman, Still, Berry & Bialek: Network Information and Connected Correlations (2003).

XOR and Spekkens Bits

We can use six (from sixteen) Spekkens[2] states to verify the compatibility of an imaginary multiway schedule with valid conditions. For example, while:

are valid pair orderings, with epistemic restrictions shown in blue,

are potentially valid ordering states, which can be specivied by (for example) a subtransaction schedule,

Critical to Spekkens' argument is that it's the epistemic states, and not the ontic ones, that resemble qubits. In this specification, we tie the two entangled toy qubits for each interaction pair (what we call MORTS – Morphism pORTS). The key challenge is being precise about what it represents (and what it doesn't) in the protocol. By creating a deliberate epistemic restriction, Spekkens' toy model has been proven to exhibit several what were previously considered 'quantum-only' behaviors. Critically (for our purposes in this specification) this includes no-cloning and teleportation. By recognizing the arrival of events (Petri tokens), we can distinguish (inside the subsructure implementation) the event orderings, but only by setting up the system to ask the right yes/no question. This sequence begins with the 'liveness architecture' described below.

I'm OK, Are You OK?

The `liveness` conversation is a tuple {I'm OK, are you OK?}. A statement about what I know about myself, and a question I am asking my neighbor. The informational states are 0, +1 and −1. We treat information as the answer to a yes/no question.

They may be captured (measured) by Alice or Bob respectively as a completion of the information transfer in the link. This may be used as an initial token transfer, registered as a loan (or borrowing) of the token in the link in one (2 phase) operation, or may be followed by a second (2 phase) operation which completes the transfer of 'ownership' of the token, such that all knowledge of the token appears on one side of the link or the other, but not both, and not neither.

While figure 2 describes the distinction between three computational entities (variables) on a single link, this can be generalized to N variables (computational entities) on a single link between two computers – or between pairs of computational entities executing in a single address space on a single multicore computer. Figure 3 shows a chain of pairs. This can clearly be generalized to a tree of pairs.

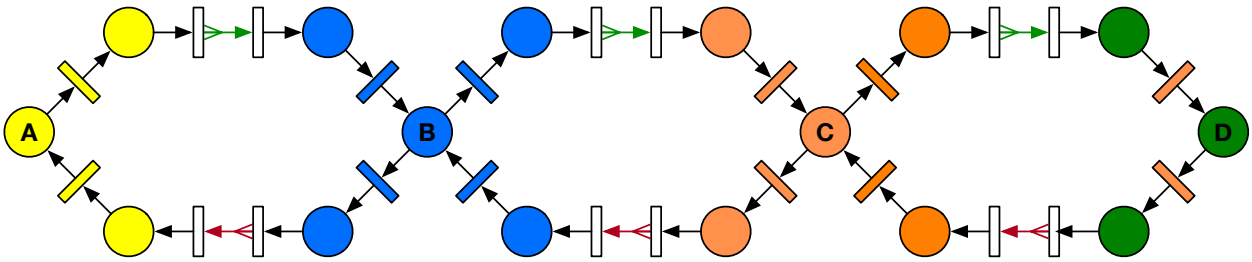

Figure 3: Four-party operation (A,B,C,D tied together with three MORTS (Morphisim pORTS)

Every pair of interactions requires four Spekkens Epistemic restriction bits. Triplets (A, B and C) shown in the above example will require eight Spekkens bits, and a chain of N computational entities in series will require $4(N-1)$ Spekkens bits for each pairwise interaction. i.e. some combinations of interactions in the connected FPGA logic will enable the selection of a compatible subset of a multiway execution.

The idea is that we can do some number of roundtrip phases in substructure (the FPGA layer), without having to go up to the X86 (or other Host) processor, over the slow PCIe bus.

[2]Robert W. Spekkens In defense of the epistemic view of quantum states: a toy theory (2004).

Spekkens Epistemic Restrictions

We model Spekkens epistemic/ontological boundary in a Petri net by two consecutive Petri transitions (the action mechanism which fires the Petri net to transfer tokens) connected in series without an intervening place, because this information cannot be observed (therefore it cannot exist and be available to the programmer). However, the epistemic information at A, B, and C can.

The following conventional Petri Net shows the problem – in execution, the net is designed with a marking of two tokens (we are ignoring for the time being that B in the middle also has it's own token).

This is a Petri net, not a state diagram, so the the tokens from P1 and P11 can circulate in parallel completely asynchronously.

This can be generalized to multiport FPGA SmartNICs, by imagining that each port on the network (e.g. 8 transceivers operating at 116Gb/s on the most recent Intel M-Series FPGA's), is a single link, or part of an $(N-1)$ Chain of links along the link axis. And there are M such links that can operate in parallel (completely concurrently in ordering the events on this multiplicity of ports).

The total number of Spekkens bits required for a cluster of 9 connected cells (the self-cell and its 8 connected neighbors), is thus $4 \times N \times M$.

Observable State (EPI)

In Figure 4 P1, P2 and P6 represent the State available to Alice. P2 and P3 are the EPI (eState) information, available only to the Link. The Link (connected status) information is available to the cell agent, along with all the P1 information for every other link.

Unobservable State (ONT)

The key aspect of this diagram is that there is no place (oState) between T_2 and T_3 (or any other pair of transitions in series, representing the `snd` and `rcv` information in the link. There is no ontological information, but according to the Spekkens' rule, we must recognize that there will be an epistemic restriction, where we can only know half the ontic information (in a pure state system).

The Petri transitions are *events*, as well as *actions* in the language of Petri Nets. One represents sending a packet, the other represents receiving the same packet. They are 'separable' because it is possible, due to some failure, that the packet was sent, but not received. This happens all the time in conventional switched networks, because dropping packets is accepted behavior of switches and routers. But even in a directly connected network, it is still possible for the link to be disconnected, or the packets to be corrupted.

The coin of the realm, throughout this architecture is *events*. Even if the packets are corrupted or torn, they still represent the arrival of information. We represent this mathematically by the low-level (black) tokens in the Petri Net, and implement them using packets going into and out of the SmartNIC.

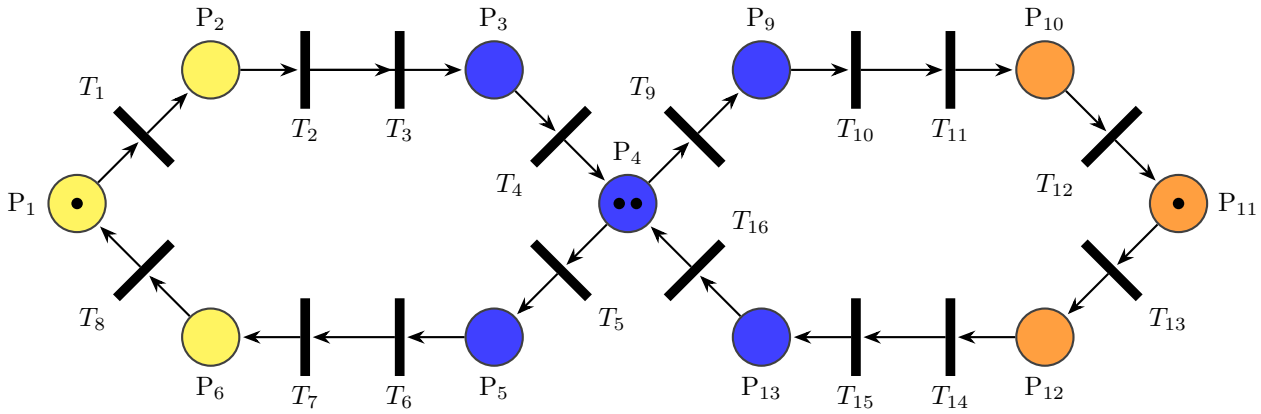

Figure 4: Petri Net Link Chain with Standard Low-level (Black) Tokens

Race Conditions ≡ Uncertainty

There is a race condition in figure 4: the black tokens at P_1 and P_{11} will circulate and sometimes collide at P_4. When they do, they will form an order of events, which can be used to validate or deny an ordering constraint specified by a subtransaction schedule.

This is where we make a distinction between the low-level Petri Net (black) tokens, which are not conserved, so we use black tokens purely to maintain liveness, and not for the EQ/CQ mechanism.

When we want EQ/CQ, we need to capture (and test/manipulate) the ordering of events from different cells, then we must use the next level of abstraction, called Colored Tokens in the literature[3]. Colored tokens are like types in data structures. In our design we use them to construct the EQ/CQ mechanism. Each Colored token must now resemble a 'qubit-like' data structure, with more than two states. The Spekken's model supports values of +1, -1, and 0. Using two classical bits, we have one more state, which we can use to represent the uninitialized. Or, noticing that the OR gives us fractional probabilities, and EOR gives us unitary +1, -1 and 0 for 'connected' information, we can utilize these two bits directly. By pairing

- The 8-bit state is used for *colored* tokens. Where each of the nodes (can specify a color for their token).
- The 16-bit state is used as colored tokens for the application layers above.

[3]Similar to the Baez, Foley & Moeller's Catalysts.

Causal Event Capture

Spekkens Event Ordering Encoding (A and B tokens only, Observed from C or D)

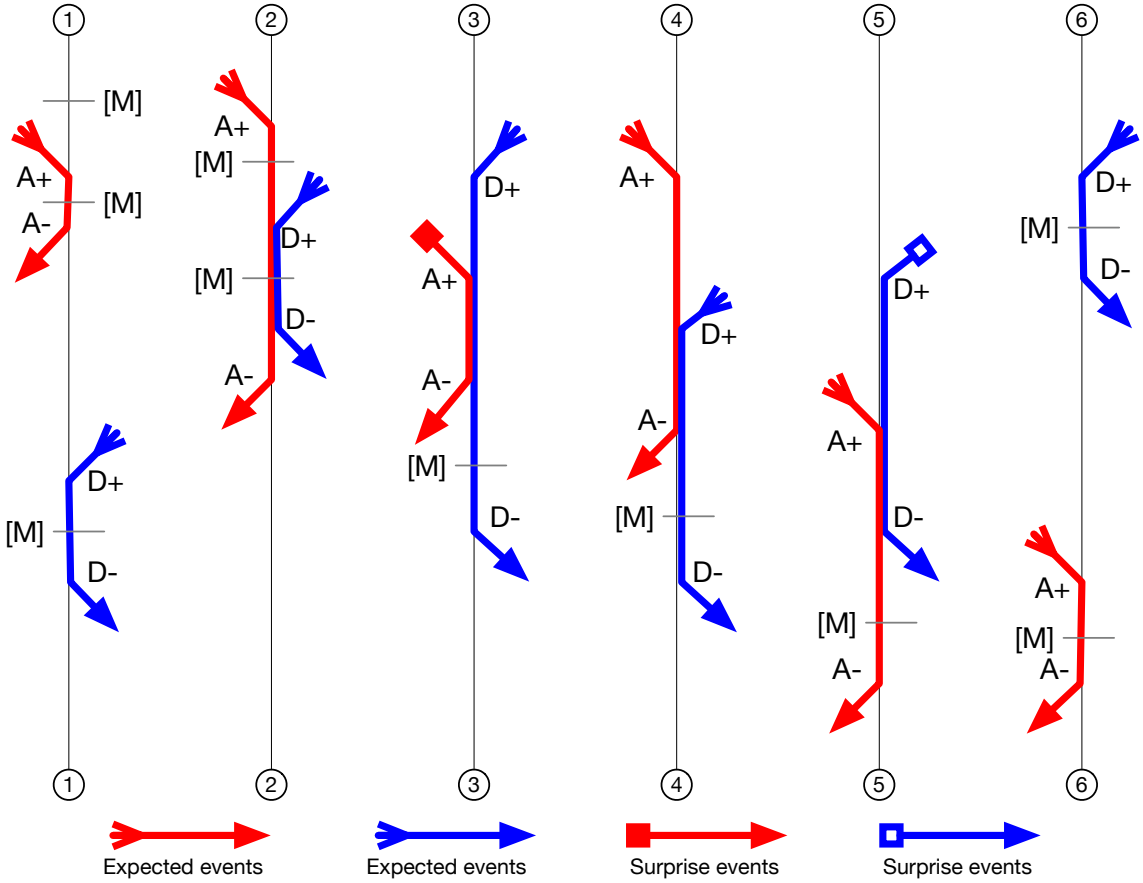

Figure 5: Petri Net Token Event orderings (Spekkens Alternatives)

The six event orderings shown in fig: 5 are the six alternatives ways the Petri Net tokens can arrive and depart in the FPGA SERDES. When measured (M) in the input slice they depict (0) no token, (+1) token arrives, and (−1) token departs. This is why we use a toy qubit[4]. We can calculate the token quantities in six different ways:

1. Alice token (A) arrives, Alice token (A) departs. Bob token (B) arrives then Bob (B) token departs.
2. A arrives (+1), B arrives (+1), B departs (−1) then A departs (−1).
3. B arrives, A arrives, A departs, B departs.
4. A arrives, B arrives, A departs, B departs.
5. B arrives, A arrives, B departs, A departs.
6. B arrives, B departs, A arrives, A departs (a causal reflection it (1) above).

We include in above diagrams - places (with 'changes' that are on their way with something like a crows feet tail on the event arrow. Crows foot tail is code for no change since last. Square is code for something changed. This can be interpreted as a sequence of events (crows foot – Petri SND has changed in the source, but the packet hasn't arrived at this node yet). And differentiate that from a the A +1 events, when that packet arrives (and changes the event structure of the measurement) at this node.

[4]See: Disilvestro & Markhaam: Quantum protocols with Spekkens Toy Model.

Recursive Epistricted States

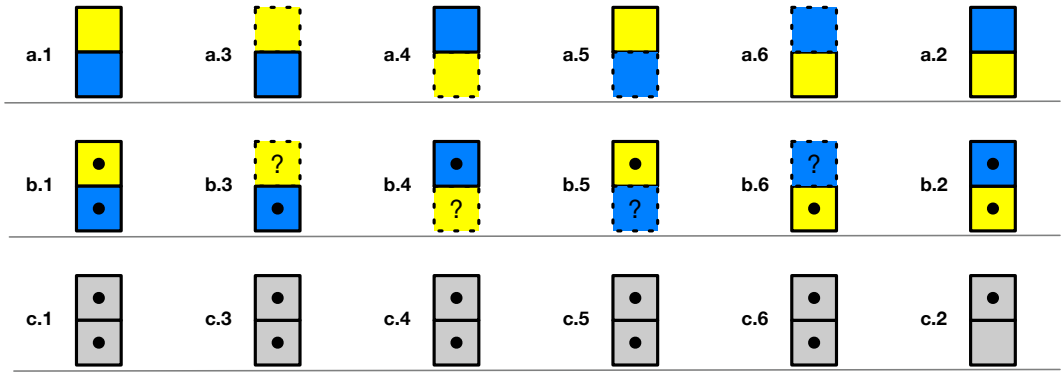

Figure 6: Another View of the Knowledge Balance principle (KBP)

Figure 6 shows another view of the six epistricted states. Columns are order, 'knowledge' resides internally in the computational agent which, without special processing, appears in 'superposition'. Row (a) shows what this universe looks like from the inside. This is Spekkens' 'knowledge:' a.1 and a.2 are the concrete knowledge of whether Alice happened before Bob, or Bob Happened before Alice. a.3, a.4 and a.5 show the possibilities with the Stop and Wait (SAW) protocol, where the solid boxes are the current state (held in SAW by this node), and the dotted boxes are where the packet has been sent and we are waiting on the SAW from the other node. Row (b) shows what we have from a low level Petri (black) token perspective. The OR function would show that the MORT is still entangled, but the "?" boxes would represent *stale* information: this 'last packet received' has already been returned, and the other agent may have already changed, and it's response could even be on it's way back to us, traveling at the speed of light in the medium. The Exclusive OR (XOR) Function would compute 'zero' as it's output. When the output is zero, 'events' are not propagated to the layer(s) above (the level 1 EPI knowledge). Row (c) shows what we can call level 2 EPI knowledge, because the hiding (confinement) of the events until there is new information), now obfuscates all knowledge, with the exception of the the knowledge that the MORT is LIVE, and its TRANSITION to NOT LIVE (all tokens drained from the MORT).

The principle of internal & external information (the outside cannot see what is going on in the inside, unless it is in the inside). The Spekkens KBP works recursively. An ontic state is a state of reality, whereas epistemic state is a stake of knowledge. We could refer to Ontic state as a Zero'th level knowledge. In KBP: Knowledge is the number of questions about the physical state of a system that are answered, must always be equal to the number that are unanswered in a state of maximal knowledge. If information is a surprise, and knowledge is what is left behind (stored in memory), then information is some kind of derivative of knowledge. If information as a surprise[5] is a highly perishable commodity: once you have stored in memory, it ceases to be information, and becomes knowledge.

The principle of Recursive Epistricted States (EPS) provides (from each independent agent interacting pairwise with another independent agent), a hierarchy of EPI vantage points, which are 'shielded' from the noise of the underlying event structures by hiding any interaction, unless it changes. There is no GEV in this system, the host processor in the superstructure (on the other side of the PCIe bus) needs to see only 'completed' interactions that meet the constraints of the subtransaction schedule it has provided. All housekeeping, including error handling and completion of the token operations, must be handled entirely within the substructure[6].

Our alternative method, described here, is to capture non-surprise events and keep them in the lowest level EPI knowledge – continuously in superposition with what might happen next – which may have actually happened at the source but the token may not have arrived at this (measurement) node yet. When change happens with a predicted question in the alphabet of types, the next packet that arrives carries this information with it. Black Petri tokens would be indistinguishable[7].

[5]The arrival events are a 'surprise' and represent new information if they are not 'expected'.

[6]No Timestamps. You cannot record numbers (certainly not real numbers because floating point numbers require an infinite number of digits after the decimal point), and vector timestamps with natural numbers are highly inefficient, and don't scale.

[7]Low-level Petri net tokens are indistinguishable; we may only be able to count the 'connected' (multi-) information.

Liveness Architecture

The Liveness packets contain the following :

- Declaration 'I'm OK'
- Question 'Are you OK?'

MORTS

Morts (Morphism pORTS) are a Data structure that keeps track of changes to itself:

1. Since the last heartbeat (mortbeat)
2. Sind the last 2 heartbeats (2 phase protocol - may be 1 round trip)
3. Since the last 4 heartbeats. (4 phase protocol)
4. Since the last 8 heartbeats.

This can all be done with XOR of the changes. (No modulo counting necessary)

Composable heartbeats

Heartbeats create events, which consume power, processing cycles and space.
Tilebeats (the entire tile 1 hops out) supported by a stacked tree.
Treebeats (the entire tree, separately on each branch).
You cannot do more than one hop on a black tree (the groundplane)
A stacked tree can have notifications n hops out (or however the equation was constructed)

4-Bit Heartbeat Architecture

Single Pair of MORTS – One Spekkens Entanglement

Heartbeats (MORTBeats) are associated with one link (a single connected pair of MORTS).

Heartbeats are generally One Physical Link at a time. The protocol is *compositional* (why we associate them with CT style morphisms, because they capture the appropriate mathematical principles). The *substructure* can forward packets without the *superstructure* being involved at all. This enables MORTS to compose (be forwarded with the same mathematical properties) at the lowest possible latency through each hop – and for liveness to be maintained with distant nodes many physical hops away. In addition to 'snake' and 'glider' (Cellular Automata functors), the liveness architecture can me maintained with the lowest possible overhead, and substructure algorithms can be performed in the confinement of the substructure without exposing the computation (or keys) to the vulnerabilities in the superstructure.

Alice Receives					**Alice Sends**			
Spekkens State In					**Spekkens State Out**			
eState					**eState**			If you received a packet, then you are entangled
Last Sent		Recent Recd		Pre Action	Out		Post Action	
0	0	0	0	Error	-	-	Stop	Stop - trigger error processing on this side.
0	0	0	1	Error	1	0	Error	zero\
0	0	1	0	Error	0	1	Error	zero
0	0	1	1	Entangled	0	0	Entangled	Valid EPI State — Alice Now has Token
0	1	0	0	Bob Start	0	1	Alice Follow	Init
0	1	0	1	Entangled	1	0	Entangled	Valid EPI State — Bob zero, Alice zero
0	1	1	0	Entangled	0	1	Entangled	Valid EPI State — Alice zero, Bob zero\
0	1	1	1	Error	1	1	Error	Alice zero, Bob transitions to send token
1	0	0	0	Error	0	0	Error	Alice zero\, Bob transitions to send token
1	0	0	1	Entangled	0	1	Entangled	Valid EPI State — Bob zero\, Alice zero
1	0	1	0	Entangled	1	0	Entangled	Valid EPI State — Alice zero\, Bob zero\
1	0	1	1	Error	1	1	Error	Alice zero\, Bob transitions to send token
1	1	0	0	Entangled	1	1	Entangled	Valid EPI State — Bob Now has Token
1	1	0	1	Error	0	1	Error	Alice has Token, Bob responds with zero
1	1	1	0	Error	0	1	Error	Alice has Token, Bob responds with zero\
1	1	1	1	Error	1	0	Error	Circulate current packet once

Figure 7: Spekkens Entangled State Table

Heartbeat Architecture and Confinement

EQ/CQ (with classical no-cloning & teleportation) supports distributed applications that require atomicity. Such an application structures do not work well through a conventional switched network.

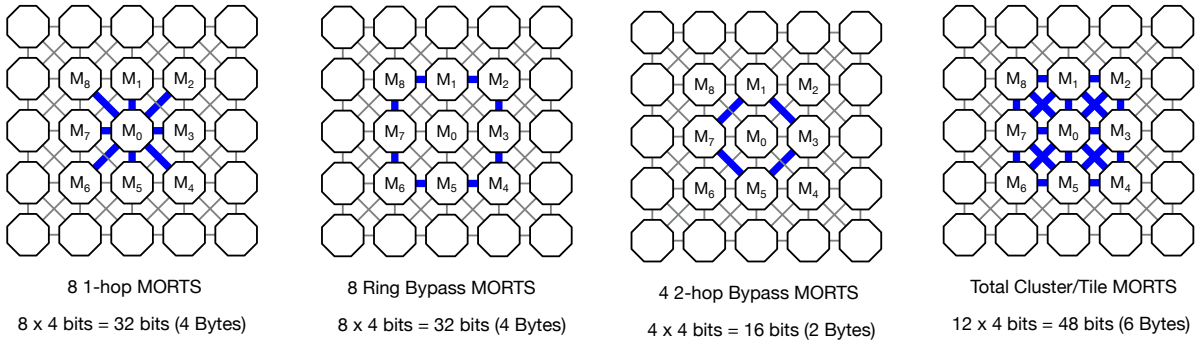

Figure 8: Cluster MORTS (blue). Local interconnect topology liveness awareness from cell (M_0)

The first use case for the heartbeat architecture is to support the 'awareness' of the local topology. Although cells may have a unique identity, they do not have a destination address which (in conventional networks) is conflated with the identity. Instead, cells can address their directly connected (1-hop) neighbors by sending and receiving messages on the port the connect to their neighbors. This gets us a 1-hop interconnect system. It is possible, but becomes rapidly more difficult to identify specific cells more than one hop away, without some kind of addressing scheme. Our Tree-Based Addressing (TBA) is a name-based addressing scheme that overcomes many of the challenges in today's networks (particularly configuration and security).

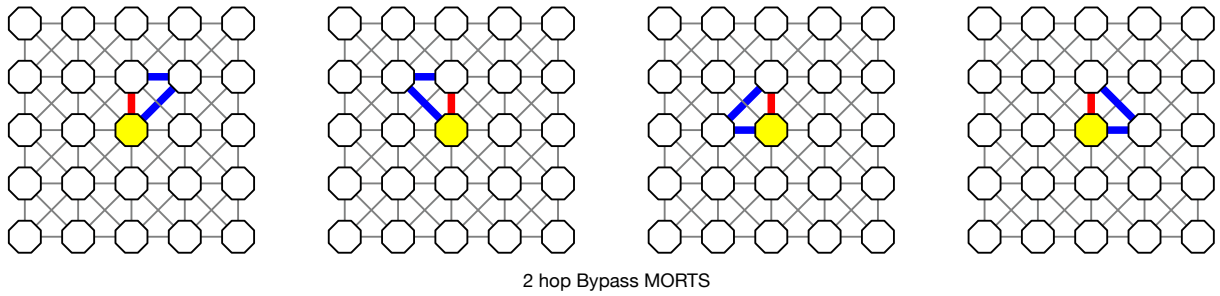

Figure 9: 2-Hop Healing MORTS (3 nodes, 3 links, 6 ports). Red link is broken, Blue links are healing.

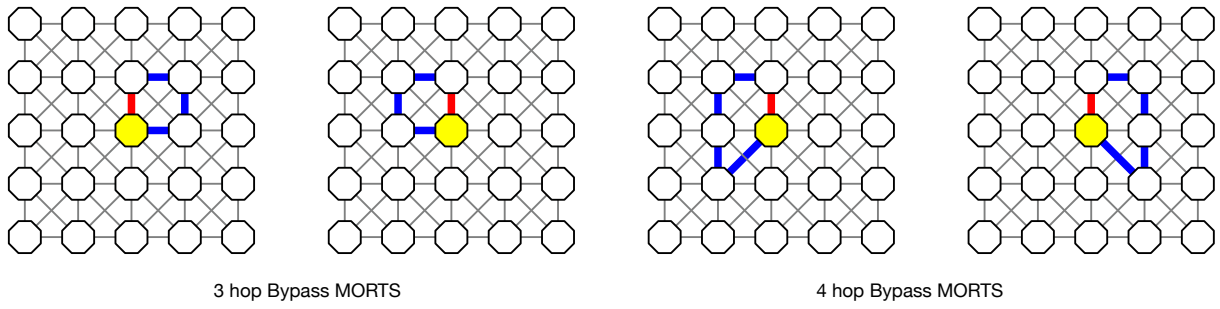

Figure 10: Red link is broken, Blue links are 3/4-Hop Healing MORTS (5 nodes, 5 links, 10 ports

MORTS provide toy Qubit Awareness on 9 cell clusters (1-hop connected) and 25 cell clusters (2-hop connected). Coherent awareness of local interconnect topology supports treebuilding, but more importantly, provides a greater repertoire of algorithms for rapid and deterministic tree healing.

Relative Tree Addressing

Conventional L2 `src` or `dst` addresses are not needed in the lowest layer (groundplane)[8]. Locality (confinement) is provided because destination addresses are implicit in the selected output port, and return addresses are implicit in the same connection. This provides an opportunity for confinement (with significant security benefits), and relative addressing out to only one or two hops. Beyond that Trees need to be stacked.

Liveness tokens are low-level Petri Net Tokens. They are neutral from an information perspective (Carry 0, or zero information). Mathematically, this is the same as the empty set.

For the heartbeat architecture, there are three types of slices. That help maintain link (MORT), Cluster, and BRANCH liveness respectively. In the heartbeat architecture, a set of slices is 64 bytes. A slice $s \in S$, is size $S/4$, i.e. 16 Bytes. Typically we send 1, 2 or 3 slices in the heartbeat engine. Slices 2 and 3 are sent *only* if they are needed, and can be pre-empted by the receiving cell, if it wants to begin a transaction as soon as possible[9]. The above figures show 3, and 5 party operations.

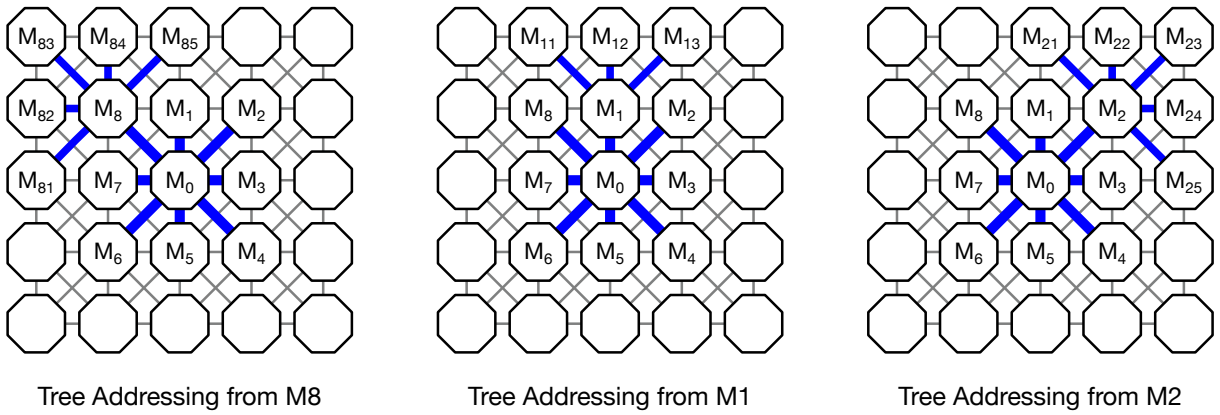

Figure 11: Tree Addressing is Relative, it looks different from different from each Tree's perspective.

Data Structures

The M cluster combines 8 1-hop directly connected MORTS, 8 Ring Bypass MORTS, and 4 2-hop Bypass MORTS. Wherever possible, we try to combine the liveness awareness bits into either 32, or 64 bit words, into 8 or 16 Byte Slices, or 64 Byte data structures that match the physical packet constraints of the packet engine, and Ethernet hardware formats.

For the host superstructure, and carrying data over the PCIe bus (or CXL) it also makes sense to align with 64-bit registers, 64-Byte cache lines, and 4K pages of 64 cache lines. This $64 \times 64 \times 64$ structure may be more than a curious coincidence.

Packet Format Requirements

The key variables (protocol fields) which characterize the above protocol are:

Headerless protocol

Shared State Machine (SSM) identifier Each protocol identifies a state machine.

Last State Describes the last state the transmitting node thinks it's Shared State Machine (SSM) it was in.

Next State Describes the next state the transmitting node wants the receiving node to move to.

Direction Ancilla The state machine 'direction' is provided in addition to the Last State / Next state. This may be used as an error check.

[8] Except for compatibility with protocols required by switches

[9] Such as what might be needed for high speed trading and other transaction systems where performance is paramount.

Slices A field indicating (a) how many slices this protocol has, and (b) which slice we are currently in.

Roundtrips A field indicating how many roundtrips are forecasted in this message set.

- Liveness (zero, zero) no data transfer.
- One-phase transfer (send / acknowledge).
- Two-phase transfer (prepare/ack, send/ack).
- Three-phase transfer (prepare, superposition, measurement)
- Four-phase transfer (two-phase borrowing transfer, followed by two-phase ownership transfer).

Slice Engine

Slice 1 a/b (8/16 Bytes) – LINK STATUS Pairwise entanglement using SAW. SWAP (8/16 B). Keyword `z` Can be sent continuously – in Stop and Wait (SAW) model. $2 \times 64 - bitwords.$ with an OPERATOR (Compare and SWAP). Topmost bit is test of all 1's (zeros in XOR of Spekkens bits).

Slice 2 a/b (16/32 Bytes) – TILE STATUS 8 ports x 4 bits = 32 bits. OPERATOR is (Compare and SWAP) 2a with 2b. Topmost bit indicates error.

Slice 3 a/b (32B/64B) BRANCH Status Contains 32B (8 x 4 bits) of BRANCH Liveness status 1-hop out. Because MORTS (like Morphisms) are transitive, it can be used in conjunction with a stacked tree to maintain liveness for the whole branch of that stacked tree.

Note: forwarding implies more than 1-hop processing. This is needed only for 'Tree Operations', 2 or more hops down the branch. Therefore we do not include it in slice 1 (link only liveness), and may not need to include it in slice 2 (1-hop cluster liveness).

[ACTION – ADD PACKET FORMAT DIAGRAMS]

ERROR Handling

Our philosophy, on receiving a corrupted, torn, or incorrect packet, is to return it to the sender with the `ERROR` flag set, so the sender can identify the cause of the problem (perhaps by comparing it to the packet it sent, and calling some higher level error routine in the upper software layers). If the sender is unable to identify the cause on their end, then it may declare the link failed, and trigger the cell agent to heal around the broken link. After the traffic has resumed on that tree, the link may be handed over to a (local) repair routine which checks the cause of the incorrect link transmission, and carries our various tests, before putting the link back into service (as a standby for next time the tree fails).

This mechanism replaces the conventional Bidirectional Link Failure detection capability in existing protocols. Because the SAW is essentially a circulating token protocol (a degenerate form of IBM's Token Ring), it is continually testing to see if the link is fully operational in both directions.

